

Assignment-2: UTXO-Based Transaction Verification

2502-SIL763/7163

March 2026

1 Objective

In this assignment, you will implement the transaction verification component of a simplified blockchain system based on the **UTXO (Unspent Transaction Output) model**.

Your implementation will validate transactions by enforcing:

- UTXO-based accounting
- Script-based locking and unlocking conditions
- Digital signature verification
- Transaction fee constraints
- Double-spend prevention

By the end of the assignment, your system should correctly determine whether a transaction is valid or invalid according to the provided rules.

2 Framework Provided

You will receive starter code containing the basic structure of the system.

The framework includes:

- Transaction data structures
- A simplified script execution engine
- Cryptographic utilities

- A UTXO management module
- Public test cases

Your task is to complete the missing verification logic in the provided modules.

3 System Overview

Many blockchain systems use the **UTXO model** to track ownership of funds. Instead of maintaining account balances, the system tracks unspent transaction outputs.

Each transaction:

- Consumes previously unspent outputs as inputs
- Creates new outputs that may be spent in future transactions

Each output contains a **locking script** specifying the conditions required to spend that output. When spending an output, a transaction provides an **unlocking script**. The combined scripts are executed to determine whether the spending condition is satisfied.

Your verifier must execute these scripts and validate transactions accordingly.

4 Your Tasks

You must implement the core transaction verification logic in the starter code.

Your implementation must:

- Verify that referenced inputs exist in the UTXO set
- Prevent double-spending
- Validate digital signatures
- Execute scripts correctly
- Verify that total input value is greater than or equal to total output value
- Reject invalid transactions

You will interact with the following modules in the starter code:

- `verifier.py` – transaction validation logic

- `script.py` – script execution engine
- `transaction.py` – transaction structure and signing helpers
- `utxo.py` – management of unspent transaction outputs
- `crypto.py` – cryptographic utilities

The starter code contains comments and TODO markers indicating where implementation is required.

5 Script Execution Model

Scripts operate using a **stack-based execution model**. Data elements are pushed onto a stack and operations manipulate these values.

For each transaction input:

- The unlocking script is executed
- The locking script of the referenced output is then executed

If script execution fails at any point, the transaction must be rejected.

6 Transaction Validation Rules

A transaction is valid only if all of the following conditions hold:

- All referenced UTXOs exist
- No input attempts to spend an already spent output
- Script execution succeeds
- Signature checks pass
- Total outputs do not exceed total inputs

If any rule fails, the transaction must be rejected.

7 Setup and Running Public Tests

The assignment materials contain two directories:

- `starter_code/` – contains the Python modules you will implement or modify
- `tests/` – contains the public test scripts and sample transactions

Before running the tests, install the required dependencies:

```
[pip | pip3] install ecdsa pycryptodome
```

To run the public tests locally, execute the following command from the assignment root directory:

```
PYTHONPATH=starter_code [python | python3] tests/public_tests.py
```

The `PYTHONPATH=starter_code` argument ensures that Python imports the modules from the starter code directory.

8 Grading and Testing

Submissions will be evaluated using an automated grading system.

The autograder will run your implementation against a set of:

- **Visible tests** provided in the assignment test package
- **Hidden tests** used during final evaluation

These tests verify correctness of:

- Transaction validation
- Script execution
- Signature verification
- Fee validation
- Double-spend prevention

9 Submission Instructions

Due Date: March 16, 2026, 12:00 PM IST

Coding Language: Python

Submit your completed assignment through **Gradescope**.

Your submission must be a compressed archive named:

`<entry_no>_A2.zip`

Example:

`2024JCS2037_A2.zip`

The archive must contain the following files **directly in the root of the zip file (not inside any folder)**:

```
<entry_no>_A2.zip
  crypto.py
  script.py
  transaction.py
  utxo.py
  verifier.py
  report.pdf
```

Submissions with additional directories (for example `A2/crypto.py`) may fail automated grading.

10 Report Requirements

You must include a short report (**1–2 pages maximum**) describing your implementation.

Your report should briefly discuss:

- Your approach to transaction verification
- How your script execution logic works
- Any design choices made during implementation
- Any challenges encountered

The report should be concise and focused.

Good luck!